Python List Project

Mini Supermarket Management System

Build a simple supermarket system using only Python lists and the concepts already learned in class.

The project should help the student practise real-world list usage without dictionaries, classes, files, or external libraries.

Project Goal

- Manage products, prices, and stock quantity.
- Create and manage a customer cart.
- Calculate cart total using loops.
- Use slicing, copying, sorting, list methods, nested lists, and matrix operations.
- Understand aliasing, id(), ==, is, mutable and immutable data types.

Starter Data

```
products = ['Rice', 'Beans', 'Oil', 'Sugar', 'Salt']

prices = [5000, 3500, 8000, 2500, 1000]

stock = [20, 15, 10, 25, 50]

cart = []
```

Each product matches the same index in the other lists.

```
products[0] # Rice
prices[0] # 5000
stock[0] # 20
```

Part 1: Display All Products

Print all products with their price and stock.

```
Expected style:

0 - Rice | Price: 5000 | Stock: 20
1 - Beans | Price: 3500 | Stock: 15
2 - Oil | Price: 8000 | Stock: 10
3 - Sugar | Price: 2500 | Stock: 25
4 - Salt | Price: 1000 | Stock: 50
```

Concepts tested: loop, enumerate(), indexing multiple lists.

```
for index, product in enumerate(products):
print(index, product, prices[index], stock[index])
```

Part 2: Access Specific Products

- Print the first product.
- Print the last product using negative indexing.
- Print the price of Oil using its index.
- Print the stock of Sugar using its index.

Concepts tested: indexing and negative indexing.

Part 3: Add a New Product

Add the new product below using append().

```
Product: Milk
Price: 4500
Stock: 12
```

```
products.append('Milk')
prices.append(4500)
stock.append(12)
```

Concept tested: append() and keeping related lists aligned.

Part 4: Insert a Priority Product

Insert Bread at index 1.

```
Product: Bread
Price: 1500
Stock: 30
```

```
products.insert(1, 'Bread')
prices.insert(1, 1500)
stock.insert(1, 30)
```

Concept tested: insert() and index positioning.

Part 5: Extend Product List

The supermarket receives multiple new products. Add them using extend().

```
new_products = ['Tea', 'Milo', 'Cornflakes']
new_prices = [1200, 3000, 4500]
new_stock = [40, 25, 18]

products.extend(new_products)
prices.extend(new_prices)
stock.extend(new_stock)
```

Concept tested: extend() for adding multiple items.

Part 6: Update Product Price

Change the price of Oil from 8000 to 8500.

The student may manually find the index of Oil and update the matching price.

```
# Example if Oil is at index 2
prices[2] = 8500
```

Concept tested: list mutation and indexing.

Part 7: Increase All Prices by 10%

Increase every product price by 10% and update the original prices list.

```
for index, price in enumerate(prices):
prices[index] = price * 1.10

print(prices)
```

Concepts tested: enumerate(), mutation, and arithmetic.

Part 8: Sort Prices

- Create a copy of the prices list and sort it from lowest to highest using sorted().
- Sort the original prices list using .sort().
- Explain the difference between .sort() and sorted() using printed output.

```
sorted_prices = sorted(prices)
print('Sorted copy:', sorted_prices)
print('Original prices:', prices)

prices.sort()
print('Original after sort:', prices)
```

Concepts tested: .sort(), sorted(), original list vs new list.

Part 9: Product Slicing

Using the products list, print the following:

- First 3 products.
- Last 3 products.
- Products at even indexes.
- Products at odd indexes.
- Products in reverse order.
- A copy of the products list.

```
print(products[:3])
print(products[-3:])
print(products[::2])
print(products[1::2])
print(products[::-1])
products_copy = products[:]
```

Concepts tested: slicing range, step size, even/odd index slicing, reversing, and copying.

Part 10: Remove Products

Task A: Using pop()

Remove the last product and print the removed product. Also remove the matching price and stock.

```
removed_product = products.pop()
removed_price = prices.pop()
removed_stock = stock.pop()

print(removed_product, removed_price, removed_stock)
```

Task B: Using del

Delete the product at index 2. Also delete the matching price and stock.

```
del products[2]
del prices[2]
del stock[2]
```

Task C: Using remove()

Remove Salt from the products list. If .index() has not been taught, give the index manually.

```
index = products.index('Salt')

products.remove('Salt')
```

```
prices.pop(index)
stock.pop(index)
```

Concepts tested: pop(), del, remove(), and keeping lists aligned.

Part 11: Customer Cart System

A customer buys Rice, Oil, and Sugar. Add the products to cart using append(). Add their prices to cart_prices.

```
cart = []
cart_prices = []

cart.append('Rice')
cart_prices.append(5000)

cart.append('Oil')
cart_prices.append(8500)

cart.append('Sugar')
cart_prices.append(2500)

print(cart)
print(cart_prices)
```

Concepts tested: append(), indexing, and parallel lists.

Part 12: Calculate Cart Total

Use a loop to calculate the total price of all items in cart_prices. Do not use sum().

```
total = 0

for price in cart_prices:
total += price

print(total)
```

Concept tested: loop and accumulator variable.

Part 13: Convert Receipt Text to List

Convert the receipt string into a list.

```
receipt = 'Rice Oil Sugar'

receipt_list = receipt.split()

print(receipt_list)
```

Concept tested: split().

Part 14: Convert Cart List to Receipt Text

Convert the cart list into a receipt text joined with hyphens.

```
cart = ['Rice', 'Oil', 'Sugar']

receipt_text = ' - '.join(cart)

print(receipt_text)
```

Concept tested: join().

Part 15: Copying and Aliasing Test

Task A: Copy

```
copied_products = products[:]
copied_products[0] = 'Copied Rice'

print(products)
print(copied_products)
```

Task B: Alias

```
alias_products = products
alias_products[0] = 'Alias Rice'

print(products)
print(alias_products)
```

Task C: Compare

```
print(products == copied_products)
print(products is copied_products)
print(products is alias_products)

print(id(products))
print(id(copied_products))
print(id(alias_products))
```

Concepts tested: copy, aliasing, ==, is, id(), and mutable lists.

Part 16: Mutable vs Immutable Mini Test

Task A: List

```
items = ['Rice', 'Beans']
same_items = items

same_items.append('Oil')

print(items)
print(same_items)
```

Question: Did both change? Why?

Task B: String

```
name = 'Rice'
same_name = name

same_name = 'Beans'

print(name)
print(same_name)
```

Question: Did the first string change? Why not?

Concepts tested: mutable list, immutable string, reassignment vs mutation.

Part 17: Nested Sales Records

Use the sales list below.

```
sales = [
['Monday', [5000, 3500, 8000]],
['Tuesday', [2500, 1000, 4500]]
]
```

- Print 3500.
- Print 4500.
- Change 1000 to 1200.
- Print the full sales list.

Concepts tested: nested lists, double indexing, mutation inside nested lists.

Part 18: Matrix Stock Addition

The supermarket has two branches. Add both matrices element-wise.

```
branch1_stock = [
[10, 20, 30],
[40, 50, 60]
]

branch2_stock = [
[1, 2, 3],
[4, 5, 6]
]

total_stock = [
[0, 0, 0],
[0, 0, 0]
]

for i in range(len(branch1_stock)):
for j in range(len(branch1_stock[i])):
total_stock[i][j] = branch1_stock[i][j] + branch2_stock[i][j]

print(total_stock)
```

Expected result: [[11, 22, 33], [44, 55, 66]]

Concepts tested: nested loops, matrix addition, range(len(...)), nested indexing.

Part 19: Find Total Stock in Matrix

Using total_stock, find the total of all numbers.

```
total_stock = [
[11, 22, 33],
[44, 55, 66]
]

total = 0

for i in range(len(total_stock)):
for j in range(len(total_stock[i])):
total += total_stock[i][j]

print(total)
```

Expected result: 231.

Concept tested: nested loops and accumulator.

Part 20: Final Report

Print a final report like this:

```
===== SUPERMARKET REPORT =====
Products Available: [...]
Prices: [...]
```

```
Stock: [...]
Customer Cart: [...]
Receipt: Rice - Oil - Sugar
Total Cart Price: 16000
==============================
```

Concepts tested: printing, variables, lists, join(), and final output formatting.

Optional Final Challenge: Build a Menu

If the student is ready, add a simple menu.

```
print('1. View Products')
print('2. Add Product')
print('3. View Cart')
print('4. Exit')
```

If the student knows while loops, make it interactive. If not, keep it simple.

Marking Guide

Area	Marks
Displays products correctly	10
Uses indexing correctly	10
Uses append, insert, extend correctly	15
Updates lists correctly	10
Uses loops and enumerate correctly	15
Uses slicing correctly	10
Uses pop, del, remove correctly	10
Uses split and join correctly	10
Understands copy, aliasing, id, is, ==	15
Handles nested lists and matrix addition	20
Final report is clear	10
Total	**135**

Student Instruction Summary

```
Build a Mini Supermarket Management System using Python lists.

Your system should:
1. Store products, prices, and stock in lists.
2. Display all products with index, price, and stock.
3. Add new products using append().
4. Insert a product at a specific position using insert().
5. Add multiple products using extend().
6. Update product prices.
7. Increase all prices by 10% using enumerate().
8. Sort prices using sort() and sorted().
9. Slice the products list in different ways.
```

```
10. Remove products using pop(), del, and remove().
11. Create a customer cart.
12. Calculate total cart price using a loop.
13. Convert receipt text to list using split().
14. Convert cart list to receipt text using join().
15. Show the difference between copying and aliasing.
16. Use id(), is, and == to compare lists.
17. Work with nested sales records.
18. Add two stock matrices element-wise.
19. Find total stock from a matrix.
20. Print a final supermarket report.
```

Final Note

This project stays within the exact concepts already taught. It avoids dictionaries, classes, files, and advanced libraries so the student can focus on mastering Python lists properly.